

Contents

1	THE GATE FRAMEWORK — MASTER GATE FILE	3
1.1	Reconciled standalone specification for proof-carrying reasoning	3
2	0. Constitutional statement	3
2.1	0.1 The core rule	3
2.2	0.2 What a machine can certify	3
2.2.1	Shell	3
2.2.2	Core	4
2.3	0.3 Verdicts	4
2.4	0.4 Promotion classes	4
2.4.1	Archive admission	4
2.4.2	Candidate admission	4
2.4.3	Promotion admission	4
3	1. Lineage reconciliation	5
3.1	1.1 Master v1.0 PDF	5
3.2	1.2 Uploaded Master v1.1 Markdown	5
3.3	1.3 C095 executable-specification line	5
3.4	1.4 Why v1.2 changes the established count	5
3.4.1	ASSEMBLY	5
3.4.2	UNIFORMITY / DOMAIN-EXTREMUM	5
3.4.3	BAND-PROVENANCE	5
4	2. Proof-object schema	6
4.1	2.1 Identity and provenance	6
4.2	2.2 Logical fields	6
4.3	2.3 Epistemic fields	6
4.3.1	Claim status	6
4.3.2	Assumption role	6
4.3.3	Warrant	7
4.3.4	Reasoning mode	7
4.4	2.4 Restriction fields	7
4.5	2.5 Evidence marks	7
4.6	2.6 Composition fields	7
4.7	2.7 Numerical fields	8
4.8	2.8 Verification and coverage fields	8
5	3. Five typed schema profiles	8
5.1	F-1 — establishment/deployment regime	8
5.2	F-2 — assembly specification	8
5.3	F-3 — uncertainty provenance	9
5.4	F-4 — common-mode certificate	9
5.5	F-5 — reasoning bridge	9
6	4. Canonical hashes and stale-PASS discipline	10
6.1	4.1 Canonical serialization	10
6.2	4.2 Edge pinning	10
6.3	4.3 Current status of HASH-DRIFT	10
7	5. Warrant and conditional-debt propagation	10
7.1	5.1 Primitive hypotheses	10
7.2	5.2 Proven-Modulo	10
7.3	5.3 Derived and Measured	11
7.4	5.4 Literature-supported claims	11
7.5	5.5 Provisional dependencies	11

8	6. Composition witnesses	11
8.1	6.1 Operation compatibility	11
8.1.1	PRODUCT	11
8.1.2	UNION	11
8.1.3	AFFINE-ASSEMBLY	12
8.1.4	INTERVAL-ENCLOSURE	12
8.1.5	EXACT-IDENTITY	12
9	7. Mandatory SCHEMA and integrity preflight	12
10	8. The 16 established gates	12
11	Gate 1 — DOMAIN	12
12	Gate 2 — ENDPOINT	13
13	Gate 3 — POLARITY	13
14	Gate 4 — MEASURE	13
15	Gate 5 — MARK	13
16	Gate 6 — COMPOSITION	13
17	Gate 7 — RUNG	13
18	Gate 8 — PRECEDENCE	13
19	Gate 9 — CORE CLOSURE	14
20	Gate 10 — MODEL	14
21	Gate 11 — COVERAGE	14
22	Gate 12 — COMMON-MODE	14
23	Gate 13 — HEURISTIC-BRIDGE	14
24	Gate 14 — ASSEMBLY	15
25	Gate 15 — UNIFORMITY / DOMAIN-EXTREMUM	15
26	Gate 16 — BAND-PROVENANCE	15
27	9. Domain-specific active gates	15
28	10. Registry semantics	16
28.1	10.1 Strict loading	16
28.2	10.2 Archive versus theorem registry	16
28.3	10.3 Root verdict	16
29	11. Operating disciplines	17
30	12. Retrodiction protocol	17
31	13. Candidate extensions	17
31.1	13.1 HASH-DRIFT as a numbered gate	17
31.2	13.2 General quantifier-order UNIFORMITY	17
31.3	13.3 SOURCE-FIDELITY	18

32 14. Worked example: conditional pairing bound	18
33 15. Worked example: service latency	18
34 16. Adoption checklist	19
35 Appendix A — v1.2 changes	19
36 Appendix B — claim-language limits	20

1 THE GATE FRAMEWORK — MASTER GATE FILE

1.1 Reconciled standalone specification for proof-carrying reasoning

Version: Master v1.2 (reconciled)

Date: 2026-07-17

Status: current successor specification

Supersedes without overwriting:

- Master v1.0 PDF;
- uploaded Master v1.1 Markdown;
- C095 executable-specification line also named v1.1.

Established machinery: one mandatory SCHEMA preflight, 16 numbered gates, five typed schema profiles, canonical dependency hashes, and explicit conditional-debt propagation.

Standalone notice. This document contains the complete conceptual and machine-contract specification. It can be implemented without any earlier Q0 file. Earlier versions remain indispensable provenance and retrodiction evidence, but they are not needed to understand or apply this version.

2 0. Constitutional statement

2.1 0.1 The core rule

Name the restriction; gate the drop; grade the bridge.

A claim is established within a regime. Reusing it outside that regime is a new claim. The reuse is permitted only when:

1. the restriction is retained; or
2. a first-class bridge node explicitly carries the source regime into the deployment regime; and
3. that bridge is itself graded, evidenced, and admitted by the dependency rules.

The framework therefore operates on three objects at once:

- **claims;**
- **dependency edges;**
- **bridges, witnesses, and certificates as claims in their own right.**

2.2 0.2 What a machine can certify

A proof object is a type. A gate is a typing rule.

Every substantive gate has two layers.

2.2.1 Shell

The shell is mechanically decidable from the declared proof object:

- required fields exist;

- domains are contained;
- measures and models match;
- marks appear in the evidence cone;
- a product points to an operation-compatible witness;
- a displayed coefficient recomputes conservatively;
- an edge pins the current canonical hash of its dependency.

2.2.2 Core

The core is the mathematics asserted by those declarations:

- the independence theorem is true;
- the change-of-measure formula is valid;
- the interval proof encloses the real function;
- the failure-region enumeration is complete;
- the cited source actually says what the node claims.

The machine certifies **tag consistency and arithmetic-contract consistency**. It does not, by itself, certify tag-to-content fidelity.

That limitation is not a defect to hide. It determines the division of labor:

Gates classify and block structurally invalid reuse. Proofs, recomputation, adversarial reimplementation, source retrieval, and separated adjudication detect content error.

2.3 0.3 Verdicts

Every applicable gate returns:

- **PASS** — its shell obligation is discharged;
- **FAIL** — promotion is blocked;
- **PROVISIONAL** — the claim remains usable only under a stated warrant cap;
- **NOT-APPLICABLE** — the operation guarded by that gate is absent.

A gate PASS never means “the theorem is true.” It means:

“The claim is consistent under its declared tags, certificates, and dependency objects, conditional on those declarations being faithful.”

2.4 0.4 Promotion classes

A graph may be admitted in three different senses:

2.4.1 Archive admission

The graph is structurally valid and hash-consistent. It may contain open, killed, superseded, provisional, or non-promotable claims for provenance.

2.4.2 Candidate admission

The selected roots pass every shell gate, although they may remain explicitly conditional or provisional according to their grade contract.

2.4.3 Promotion admission

The roots pass every shell gate at their stated warrant. A **Proven-Modulo** root may promote only as an explicitly conditional theorem whose complete condition set is displayed. Unconditional promotion requires an empty condition set.

These states must not be collapsed into one word such as “admitted.”

3 1. Lineage reconciliation

Three partially divergent artifacts preceded this version.

3.1 1.1 Master v1.0 PDF

The PDF introduced the restriction-drop framework, 13 gates, and F-1. It overstated the framework as “mechanically checkable,” treated bridges largely as free-text attestations, placed assumptions, warrants, and terminal statuses on one ladder, and contained a dangling PROVISIONAL reference.

It is preserved as the origin document, not the current specification.

3.2 1.2 Uploaded Master v1.1 Markdown

The uploaded Markdown made the decisive conceptual repairs:

- shell/core honesty;
- “gates classify; disciplines detect”;
- bridges and witnesses as graded dependency nodes;
- Derived as self-tier dependable;
- content-hash pinning and stale-PASS discipline;
- a second-domain worked example;
- candidate HASH-DRIFT and UNIFORMITY rules.

Those repairs are retained here.

3.3 1.3 C095 executable-specification line

The C095 line separately added machine profiles needed by recorded arithmetic failures:

- SCHEMA preflight;
- establishment and deployment regimes;
- ASSEMBLY;
- DOMAIN-INFIMUM;
- BAND-PROVENANCE;
- certificate objects for common-mode and coverage;
- a second-domain executable contract.

Those additions are integrated here rather than maintained as a competing v1.1.

3.4 1.4 Why v1.2 changes the established count

The uploaded v1.1 correctly refused to mint new gates without retrodiction. Subsequent frozen failures supplied the evidence:

3.4.1 ASSEMBLY

- an upper display rounded (4.35) downward to (4.3);
- a first-moment coefficient (0.8411) was displayed before applying subunit AO and Bonferroni losses.

3.4.2 UNIFORMITY / DOMAIN-EXTREMUM

- sampled upper rungs were used toward a continuous finite-range claim without between-rung control;
- the sampled Bonferroni threshold exceeded 80 while the full-domain limit was (79.9889203915...).

3.4.3 BAND-PROVENANCE

- an unexplained (± 0.02) band entered an upper theorem;
- a randomized orthant diagnostic changed between releases because its algorithmic state was not frozen.

These are independent enough, selective over the failure record, and not mechanically discharged by the original gates. Gates 14–16 are therefore established in v1.2.

4 2. Proof-object schema

Every externally visible claim is an atomic proof object.

4.1 2.1 Identity and provenance

field	obligation
<code>claim_id</code>	globally unique, stable identifier
<code>statement</code>	one atomic assertion
<code>kind</code>	statement, assumption, definition, measurement, bridge, witness, or certificate
<code>source_precedence_id</code>	source/supersession lineage
<code>evidence_ids</code>	immutable proof, computation, measurement, or source objects
<code>supersedes</code>	claim IDs replaced by this object
<code>alias_of</code>	explicit content alias, if any
<code>content_hash</code>	canonical SHA-256 of every load-bearing field
<code>status</code>	live, open, killed, superseded, or retired

A human label such as `C#1` is not a content hash.

4.2 2.2 Logical fields

field	obligation
<code>quantifier</code>	exact, for-all, exists, asymptotic, typical, measured-at-rung, or conditional
<code>quantifier_order</code>	explicit when two or more quantifiers interact
<code>direction</code>	upper, lower, equality, structural, or none
<code>bound_coefficient</code>	displayed coefficient for a numerical bound
<code>conditional_on</code>	complete finite set of unresolved conditions
<code>closed_core</code>	whether the claim asserts an unconditional finished core

4.3 2.3 Epistemic fields

The following are independent axes.

4.3.1 Claim status

- LIVE
- OPEN
- KILLED
- SUPERSEDED
- RETIRED

4.3.2 Assumption role

- NONE
- DEFINITION
- PRIMITIVE-HYPOTHESIS
- POLICY-THRESHOLD

4.3.3 Warrant

- PROVEN
- PROVEN-MODULO
- CERTIFIED
- DERIVED
- LITERATURE-SUPPORTED
- MEASURED
- PLAUSIBLE
- CONJECTURE

4.3.4 Reasoning mode

- DEDUCTIVE
- EMPIRICAL
- ANALOGICAL
- HEURISTIC
- NORMATIVE

An assumption role is not a warrant. An open or killed status is not a weak warrant.

4.4 2.4 Restriction fields

Every claim carries both sides of reuse:

- `establishment_regime`;
- `deployment_regime`.

A regime is the tuple

```
(  
  exact model,  
  base measure or conditioning law,  
  parameter domain,  
  spatial/state-space region,  
  scale/rung,  
  conditioning depth,  
  marks/types,  
  numerical or deployment mode  
)
```

The parent is checked against the dependency's **establishment** regime, not against whatever broad regime the dependency author later writes into the parent.

4.5 2.5 Evidence marks

- `required_marks` describe the estimand.
- `supplied_marks` describe what an evidence node actually controls.

For a non-leaf claim, marks are collected from the evidence/dependency cone. A parent cannot satisfy MARK merely by copying its own required marks into its own `supplied_marks` field.

4.6 2.6 Composition fields

Every non-atomic operation declares its operation type:

- PRODUCT
- UNION
- AFFINE-ASSEMBLY
- MONOTONE-TRANSFORM
- INTERVAL-ENCLOSURE

- EXACT-IDENTITY

It points to a first-class witness node whose witness type is compatible with that operation.

4.7 2.7 Numerical fields

A numerical proof object may carry:

- measured values with scale, parameter, and evidence tags;
- endpoint certificates;
- an F-2 assembly specification;
- a full-domain extremum certificate;
- an uncertainty-provenance certificate;
- active domain-specific gates.

4.8 2.8 Verification and coverage fields

Agreement-based promotion identifies:

- the instruments;
- shared code/data/preprocessing;
- the error channel under test;
- the disjoint-error argument or external crosscheck.

Coverage assertions identify:

- the possible-failure universe currently registered;
 - chart/detector IDs per region;
 - an uncovered-mass bound;
 - any total-risk target.
-

5 3. Five typed schema profiles

5.1 F-1 — establishment/deployment regime

A bridge is needed whenever a deployment regime differs from the establishment regime in model, measure, domain, scale, conditioning depth, region, or marks.

Every bridge node declares:

```
bridge kind
source regime
target regime
scope domain
transferred marks, where applicable
scope statement
falsifier
```

A generic “change of measure” label does not bridge arbitrary measures.

5.2 F-2 — assembly specification

Every displayed assembled number records:

```
canonical expression
named inputs
source ID of each input
monotonicity sign of each input
required terms
```

positive upper residuals
negative lower losses
claim direction
rounding direction
working precision
displayed value

Every required or residual term must be both present and consumed by the expression.

5.3 F-3 — uncertainty provenance

Every load-bearing band records:

kind
side
construction
coverage domain
confidence level, where statistical
sample size
multiplicity scope
seed or algorithmic state
deterministic tolerance
side consumed by the parent theorem
source hash

A forecast is not a theorem band.

5.4 F-4 — common-mode certificate

Every agreement-based promotion records:

instrument IDs
shared code
shared data
shared preprocessing
shared model assumptions
shared numerical libraries
error channel under test
disjoint-error argument
external contradiction test
source hash

A Boolean such as `error_independence_arg=true` is not a certificate.

5.5 F-5 — reasoning bridge

A load-bearing analogical or heuristic step records:

source domain
target domain
scope
bridge claim ID
proof hash
falsifier
grade cap without the bridge

6 4. Canonical hashes and stale-PASS discipline

6.1 4.1 Canonical serialization

The content hash is

$$\text{SHA256}(\text{CanonicalJSON}(\text{all load-bearing fields})).$$

Canonical JSON uses:

- sorted keys;
- deterministic separators;
- normalized enum values;
- deterministic ordering of sets and maps;
- no `content_hash` field inside the hashed payload.

The payload includes dependency edges and each edge's pinned target hash.

6.2 4.2 Edge pinning

Every edge stores the canonical content hash of its target at the time the parent was gated.

If the target changes without a supersession event, the parent becomes stale and must be re-gated.

6.3 4.3 Current status of HASH-DRIFT

Hash integrity is mandatory in the SCHEMA/integrity preflight in v1.2.

It remains uncanceled as an independent numbered gate until the frozen failure record supplies the two independent stale-parent incidents demanded by the retrodiction protocol. Implementing an integrity invariant does not require inflating the numbered gate count.

7 5. Warrant and conditional-debt propagation

7.1 5.1 Primitive hypotheses

A primitive hypothesis does not become true by being labeled primitive.

It may support:

- a **Proven-Modulo** theorem whose statement displays it as a condition;
- a Derived or Plausible investigation whose condition ledger displays it.

It cannot support an unconditional **Proven** root.

7.2 5.2 Proven-Modulo

Proven-Modulo grades an implication:

$$H_1, \dots, H_m \implies C.$$

Its condition set is part of the theorem object. Any parent that uses the result non-conditionally must either:

1. discharge all conditions; or
2. become **Proven-Modulo** and inherit the complete condition union.

Conditions cannot disappear by passing through an intermediate node.

7.3 5.3 Derived and Measured

Derived is self-tier dependable:

- Derived-on-Derived is allowed;
- a Derived dependency cannot support a Certified or Proven parent non-conditionally.

A calculation using measured inputs carries the measured weakest link unless a statistical/interval theorem promotes those inputs.

Independent numerical reproduction normally promotes a result to **Certified**, not automatically to referee-grade **Proven**.

7.4 5.4 Literature-supported claims

A literature-supported theorem may support a high-grade parent only when:

- the cited source supports the exact statement;
- model, measure, marks, and domain match; or
- a graded adaptation bridge is present.

“Published somewhere” is not a universal transfer certificate.

7.5 5.5 Provisional dependencies

A provisional result carries an effective warrant cap. Every non-conditional parent inherits the cap.

A parent cannot evade the cap by reading the dependency’s declared grade while ignoring its gate verdict.

8 6. Composition witnesses

Witnesses are graded nodes.

8.1 6.1 Operation compatibility

8.1.1 PRODUCT

Allowed witnesses:

- independence;
- conditional independence;
- Markov property plus the required transition theorem;
- negative dependence in the needed direction;
- comparison theorem;
- direct joint certificate;
- exact algebra.

8.1.2 UNION

Allowed witnesses:

- union bound;
- direct joint certificate;
- exact algebra.

A union bound requires neither independence nor disjointness.

8.1.3 AFFINE-ASSEMBLY

Allowed witnesses:

- monotone assembly;
- outward interval arithmetic;
- exact algebra.

8.1.4 INTERVAL-ENCLOSURE

Allowed witnesses:

- outward interval arithmetic;
- direct joint certificate.

8.1.5 EXACT-IDENTITY

Allowed witness:

- exact algebra.

A union-bound witness cannot license a product.

9 7. Mandatory SCHEMA and integrity preflight

Before Gate 1:

1. parse the exact supported schema version;
2. verify every claim ID is unique;
3. verify every target exists;
4. verify the graph is acyclic;
5. verify kind-specific fields;
6. recompute every content hash;
7. verify every pinned edge hash;
8. verify status/warrant consistency;
9. verify the selected roots exist;
10. reject partial registry loading;
11. reject bundled multi-assertion claims where atomicity is violated;
12. record every refusal rather than silently omitting the claim.

A malformed registry load is all-or-nothing.

10 8. The 16 established gates

11 Gate 1 — DOMAIN

Guards: parameter-range containment.

Shell: every load-bearing dependency explicitly carries every free parent parameter and its establishment domain contains the complete deployment domain.

Missing domain metadata is a failure for a quantified parent, not a silent skip.

Bridge: a typed domain-transfer node whose target scope contains the parent domain.

12 Gate 2 — ENDPOINT

Guards: declared boundaries.

Every finite-range numerical claim carries endpoint certificate IDs. A Boolean written directly into a registry is shell evidence only; endpoint fidelity still requires an independently executed certificate.

Endpoint PASS is necessary, never sufficient for a uniform claim.

13 Gate 3 — POLARITY

Guards: sidedness and loss propagation.

- upper claims consume upper inputs and add every positive residual;
- lower claims consume lower inputs and apply every subunit multiplicative or subtractive loss;
- a symmetric band is consumed on the unfavorable side.

14 Gate 4 — MEASURE

Guards: base law and conditioning.

A mismatch requires a bridge node whose source measure, target measure, and scope domain exactly match the edge being bridged.

15 Gate 5 — MARK

Guards: auxiliary types and estimands.

Required marks are collected from evidence dependencies. A parent cannot self-attest its evidence marks. Missing marks require a typed mark-transfer node that explicitly lists the transferred marks.

16 Gate 6 — COMPOSITION

Guards: the license to combine.

Every non-atomic operation points to an operation-compatible, sufficiently graded witness node.

17 Gate 7 — RUNG

Guards: measured scale and parameter tags.

Every measured value carries:

- scale;
- parameter/rung tags;
- evidence ID.

Cross-scale use requires a typed scale bridge.

18 Gate 8 — PRECEDENCE

Guards: live/dead lineage.

The root itself and every reachable node must be live for a live promoted theorem. Killed, superseded, or retired roots fail even when they have no descendants.

19 Gate 9 — CORE CLOSURE

Guards: finished/open boundary.

A closed core has:

- no Open status;
- no Plausible or Conjecture dependency;
- no explicit conditional debt;
- no provisional gate;
- no open domain-specific active gate.

A conditional theorem may be valid without being a closed core.

20 Gate 10 — MODEL

Guards: exact mathematical or probabilistic model.

A transfer certificate declares the exact source model, target model, and scope. A generic high-grade node cannot bridge arbitrary model pairs.

21 Gate 11 — COVERAGE

Guards: charted failure space.

The shell checks the registered failure universe. The core remains whether the universe is complete.

A PASS means:

“No gap was found in the currently registered universe.”

It never means:

“No unregistered failure mode exists.”

22 Gate 12 — COMMON-MODE

Guards: shared verification error.

Agreement-based promotion requires a first-class certificate identifying the instruments and the error channel under test.

- disjoint-error proof or external contradiction test: PASS;
- shared/unknown channel: PROVISIONAL;
- no certificate: PROVISIONAL.

The provisional cap is at most **Derived** unless a stricter domain policy is declared. It is not “the claim’s current grade,” because that would preserve an already inflated label.

Meta-validation status: operationally established; the historical retrodiction claims for this gate remain provisional until their frozen failure IDs and hashes are supplied.

23 Gate 13 — HEURISTIC-BRIDGE

Guards: analogy-to-derivation transition.

A load-bearing analogy supporting a quantitative or asymptotic claim requires a typed bridge node with scope, derivation evidence, and a falsifier.

Without it:

- a claim above Plausible fails;

- a Plausible/Conjecture claim remains provisional.

Meta-validation status: the same evidence caveat as COMMON-MODE applies.

24 Gate 14 — ASSEMBLY

Guards: arithmetic propagation and conservative display.

PASS requires:

1. the canonical expression evaluates;
2. every input has a source;
3. every named required term is present and used;
4. every positive upper residual is present and used;
5. every negative lower loss is present and used;
6. monotonicity directions are declared;
7. upper values round up;
8. lower values round down;
9. equality displays match exactly or by outward interval.

Canonical retrodiction cases include the (4.3/4.35) upper display and the unpropagated (0.8411) lower coefficient.

25 Gate 15 — UNIFORMITY / DOMAIN-EXTREMUM

Machine alias: DOMAIN-INFIMUM.

Guards: a uniform coefficient or quantifier over the entire parameter domain.

For numerical bounds, accepted certificates include:

- exact analytic extremum;
- monotonicity plus endpoint;
- outward interval or Taylor model;
- exhaustive finite enumeration;
- validated grid plus proved modulus;
- asymptotic limit plus monotonicity and compact remainder.

Sampled rungs are not a uniformity certificate.

For general logical claims, quantifier order is recorded explicitly. The broader non-numerical quantifier-order profile remains a candidate extension until it obtains its own retrodiction record; the numerical uniformity gate is established now.

26 Gate 16 — BAND-PROVENANCE

Guards: the construction and interpretation of uncertainty.

A load-bearing band must record all metadata applicable to its type. Randomized numerical algorithms must freeze their seed or integration state and a deterministic tolerance.

A symmetric band inside an upper theorem is consumed on its upper side; inside a lower theorem, on its lower side.

27 9. Domain-specific active gates

A general kernel may carry named project gates such as:

BR-REP
BR-MARK
H4-PATH

U-SHAPE
U-UNCERTAINTY-CAL
TORUS-GLOBAL-PARTITION
UB_G_RESIDUAL_UNIFORM

An active gate is a first-class contract field with status:

- CLOSED
- OPEN
- KILLED
- BLOCKED-EXTERNAL
- NOT-CLAIMED

A load-bearing OPEN or BLOCKED gate blocks promotion unless the parent is explicitly formulated as a conditional theorem and the gate is represented as a named condition object.

28 10. Registry semantics

A registry is not merely a dictionary of claims.

It contains:

schema version
root IDs
release metadata
claims in a valid DAG
canonical hashes
edge-pinned hashes
optional admission report

28.1 10.1 Strict loading

Loading must:

1. reject an unsupported schema;
2. reject missing targets;
3. reject cycles;
4. reject content-hash mismatch;
5. reject stale edges;
6. reject invalid status/warrant combinations;
7. reject partial admission;
8. return all errors together where practical.

28.2 10.2 Archive versus theorem registry

A file may be a schema demonstration rather than a mathematical theorem registry. Its metadata must say so.

A toy coefficient, broad placeholder domain, or source-declared “Proven” label cannot be silently identified with a frozen Q0 theorem.

28.3 10.3 Root verdict

The primary root report separately displays:

shell_valid
conditional_promotable
unconditional_promotable
closable
effective_warrant

condition set
provisional gates
failed gates

A root with open conditions must never print a bare “PASS at Proven-Modulo” without displaying those conditions in the same verdict.

29 11. Operating disciplines

The machine shell presupposes:

- freeze before execute;
 - supersede, never overwrite;
 - preserve every failure;
 - write the ledger after adjudication;
 - use adversarial reimplementation;
 - separate generation, verification, and adjudication;
 - verify external citations in parallel with internal consistency;
 - persist every theorem-input seed;
 - distinguish diagnostic randomness from theorem-input randomness;
 - verify releases in ZIP and true clean-extraction modes;
 - maximize the weakest link;
 - never promote a gate from a favorable hand-selected test set.
-

30 12. Retrodiction protocol

A candidate gate qualifies only when:

1. it fires on at least two independent frozen failures;
2. COMMON-MODE analysis shows those failures are not duplicate manifestations of one cause;
3. it is silent on most of the frozen record;
4. no established gate or mandatory preflight already discharges it;
5. the evidence table names IDs, prior catch mechanisms, and artifact hashes.

The survivorship limitation is binding: this protocol sees only failures that were found.

31 13. Candidate extensions

31.1 13.1 HASH-DRIFT as a numbered gate

Hash drift is already a mandatory preflight invariant. It becomes a separately numbered gate only if the failure ledger supplies two independent cases in which a parent PASS was improperly retained after silent dependency mutation.

31.2 13.2 General quantifier-order UNIFORMITY

Gate 15 establishes numerical uniformity. A broader gate for logical quantifier swaps such as

$$\forall \varepsilon \exists \delta \not\Rightarrow \exists \delta \forall \varepsilon$$

remains staged until its own failure evidence clears retrodiction.

31.3 13.3 SOURCE-FIDELITY

No gate can decide whether a source really proves its tag from metadata alone. Source fidelity remains an operating discipline, not a candidate automatic gate, unless a domain supplies machine-checkable proof objects or formal citations.

32 14. Worked example: conditional pairing bound

Suppose the root claims

$$P_{\text{pair-Palm}}(\text{defect}) \leq Cr^3, \quad 0 < r \leq r_0, \quad b \in B.$$

The graph includes:

- inner, far, and boundary support claims;
- a union-bound witness node;
- a typed change-of-measure node for every pinned input;
- a marked-density transfer node;
- a coefficient assembly;
- a full-domain supremum certificate;
- a one-sided uncertainty certificate;
- explicit unresolved conditions.

The root promotes as **Proven-Modulo** only when:

1. every condition is named in the statement and condition set;
2. all non-conditional support is sufficiently dependable;
3. every bridge is source/target typed;
4. marks are supplied by evidence;
5. the coefficient recomputes conservatively;
6. the full-domain supremum is below the display;
7. uncertainty provenance is complete.

It is unconditional only after the condition set becomes empty.

33 15. Worked example: service latency

Claim:

Median warm-cache request latency is at most 40 ms for every arrival rate up to 2,000 requests/s on configuration C.

The schema maps:

- model $\rightarrow$ instance type, operating-system and dependency versions, service configuration;
- measure $\rightarrow$ warm-cache steady-state workload law;
- domain $\rightarrow$ arrival rate and payload size;
- rung $\rightarrow$ milliseconds at p50, not p95;
- marks $\rightarrow$ request class and cache state;
- composition $\rightarrow$ end-to-end latency witness;
- band $\rightarrow$ one-sided simultaneous benchmark uncertainty;
- uniformity $\rightarrow$ no untested mid-range queue-saturation spike.

Two load generators agreeing is provisional if they share the same clock or warmup path.

This example uses the same gates without weakening them into metaphors.

34 16. Adoption checklist

Before promoting a root, answer:

1. SCHEMA — is every required field present?
 2. HASH — does canonical SHA-256 recompute?
 3. EDGE PINNING — do all dependency hashes match?
 4. DOMAIN — does every dependency cover every parent parameter?
 5. ENDPOINT — are boundary certificates present and passing?
 6. POLARITY — are all sides, residuals, and losses conservative?
 7. MEASURE — are conditioning changes typed and graded?
 8. MARK — do evidence nodes actually carry every mark?
 9. COMPOSITION — is the witness operation-compatible?
 10. RUNG — does every measured value carry scale and evidence?
 11. PRECEDENCE — is the root or any descendant dead?
 12. CORE CLOSURE — are there unresolved debts or gates?
 13. MODEL — is every model transfer exact and scoped?
 14. COVERAGE — what registered failure region remains uncharted?
 15. COMMON-MODE — do checking instruments share the tested error channel?
 16. HEURISTIC-BRIDGE — is every load-bearing analogy derived or downgraded?
 17. ASSEMBLY — does the displayed number recompute in the conservative direction?
 18. UNIFORMITY — is the complete domain certified, not merely sampled?
 19. BAND-PROVENANCE — can the band be reconstructed?
 20. CONDITIONS — does every unresolved dependency appear in the root statement?
 21. WARRANT — does every parent respect dependency ceilings?
 22. RELEASE — does the graph pass from a true clean extraction?
-

35 Appendix A — v1.2 changes

change	reason
unified the two v1.1 lineages	prevent competing “current” specifications
retained shell/core distinction	avoid claiming metadata proves mathematics
retained graded bridge nodes	close witness/bridge laundering
split status, warrant, assumptions, reasoning mode	remove type confusion
added canonical SHA-256	human labels did not detect content mutation
made domain checking strict	absent parameters had been silently skipped
made MARK evidence-cone based	parent self-attestation was unsound
typed bridge source and target regimes	arbitrary high-grade nodes had bridged arbitrary mismatches
made witnesses operation-specific	union witness could license a product
propagated provisional ceilings	provisional dependencies had laundered into Proven parents
propagated condition sets	Proven-Modulo conditions had disappeared through parents
distinguished archive/candidate/promotion	storage had been confused with theorem promotion
added ASSEMBLY	two frozen arithmetic-propagation failures
promoted numerical UNIFORMITY	two full-domain failures
added BAND-PROVENANCE	unexplained band and randomized diagnostic
retained HASH-DRIFT as preflight/candidate	integrity is mandatory; gate-count promotion still awaits retrodiction
added strict registry load	v1.2 silently returned partial registries

change	reason
removed hard-coded output paths	executable portability
classified the uploaded Q0 registry as a demo	its R0 and coefficient are not canonical Q0 objects

36 Appendix B — claim-language limits

Do not say:

- “a gate PASS proves the theorem”;
- “the kernel checks mathematical truth”;
- “two agreeing implementations certify correctness” without COMMON-MODE;
- “a measured coefficient is scale free”;
- “a theorem is uniform” from sampled rungs;
- “a bridge is proved” because a bridge label exists;
- “the uploaded Q0 registry is the Q0 Rate Program theorem registry”;
- “R0” without disambiguating the canonical Q0 Sard condition from the legacy demo remainder condition.

Say instead:

“The proof object passes the executable shell under the declared tags and certificates. Mathematical fidelity remains grounded in the cited proofs, computations, measurements, and independent review.”

End of Gate Framework Master v1.2.